

Course Project: Volatility Analysis Using Nonparametric and Semiparametric Regression

Moinak Nath

2025-06-05

The aim of this project is to analyze the volatility curve of Apple call options at a fixed time to expiry against “moneyness.” In finance call options give the buyer the right but not the obligation to buy a stock at a specific price K by time T . The Black-Scholes and other models predict a flat volatility curve versus moneyness, which is the ratio of the strike price and the current stock price: $M = \frac{K}{S_t}$

We may write the price of an option (p) in any options pricing model as a function

$$p = C(t, T, K, S_t, \sigma, r)$$

where σ is called volatility and essentially measures the standard deviation of the stock’s return. The implied volatility for a given option price is $\sigma_i = C^{-1}(p, t, T, K, S_t, r)$

Thus, the theoretical claim is that σ_i is fixed as a function of $\frac{K}{S_t}$ if we fix time and all other parameters. We aim to test this claim by fitting a nonparametric and semi-parametric model to obtain the true relation between implied volatility and moneyness.

In practice, what one gets is a U-shaped curve known as the “volatility smile.” We seek to observe and study this phenomenon in the real world.

0) Packages

Installs and loads required packages if not already installed

```
packages <- c(
  "dplyr", "lubridate",
  "quantmod", "RQuantLib",
  "sm", "np",
  "KernSmooth", "mgcv", "forecast"
)

installed <- rownames(installed.packages())
for (pkg in packages) {
  if (!(pkg %in% installed)) install.packages(pkg)
}

suppressPackageStartupMessages({
  invisible(lapply(packages, library, character.only = TRUE))
})
```

1) Data Acquisition and Preparation

The following code was used to collect the data. It gets options data using the quantmod package, which uses Yahoo Finance as its data source. This code searches several tickers for the most data available and cleans the data by removing NA values. Obtaining financial data is often challenging, so this was a significant component of my project. I was able to obtain a dataset consisting of 53 observations (53 different options at fixed time to expiry). Results are saved in dataframe “smile_data_final”

(a) Data Collection Using QuantMod

```
library(quantmod)
library(dplyr)
library(lubridate)

get_best_options_data <- function() {
  tickers <- c("SPY", "QQQ", "IWM", "XLF", "AAPL")
  best_data <- NULL
  best_score <- 0
  best_ticker <- ""

  for(ticker in tickers) {

    tryCatch({
      current_price <- as.numeric(getQuote(ticker)$Last)

      option_chain <- getOptionChain(ticker)

      if(!is.null(option_chain$calls) && !is.null(option_chain$puts)) {
        calls <- option_chain$calls
        puts <- option_chain$puts

        calls$option_type <- "call"
        calls$expiry_date <- as.Date(calls$Expiration)
        puts$option_type <- "put"
        puts$expiry_date <- as.Date(puts$Expiration)

        all_options <- rbind(calls, puts)

        all_options$time_to_exp <- as.numeric(all_options$expiry_date - Sys.Date()) / 365.25
        all_options$mid_price <- (all_options$Bid + all_options$Ask) / 2
        all_options$moneyness <- all_options$Strike / current_price
        all_options$implied_vol <- all_options$IV

        valid_options <- all_options %>%
          filter(
            !is.na(mid_price) & mid_price > 0,
            !is.na(implied_vol) & implied_vol > 0 & implied_vol < 5,
            !is.na(moneyness) & moneyness >= 0.5 & moneyness <= 2.0,
            time_to_exp > 0
          )

        if(nrow(valid_options) > 0) {
          moneyness_range <- max(valid_options$moneyness) - min(valid_options$moneyness)
          score <- nrow(valid_options) * moneyness_range
        }
      }
    }, error = function(e) {})
```

```

        if(score > best_score) {
          best_data <- valid_options
          best_score <- score
          best_ticker <- ticker
        }
      }
    }

  }, error = function(e) {
    cat("Failed for", ticker, ":", e$message, "\n")
  })
}

if(!is.null(best_data)) {
  cat("Selected ticker:", best_ticker, "\n")
  cat("Moneyness range:", round(min(best_data$moneyness), 3), "to", round(max(best_data$moneyness), 3), "\n")

  return(list(data = best_data, ticker = best_ticker))
} else {
  cat("No valid data found!\n")
  return(NULL)
}
}

prepare_analysis_data <- function(options_data, ticker) {

  current_price <- mean(options_data$Strike / options_data$moneyness, na.rm = TRUE)

  if(sum(!is.na(options_data$moneyness)) == 0) {
    cat("ERROR: All moneyness values are NA!\n")
    return(NULL)
  }

  clean_data <- options_data %>%
    filter(
      !is.na(moneyness) & !is.infinite(moneyness),
      !is.na(implied_vol) & !is.infinite(implied_vol),
      moneyness > 0 & moneyness < 10, # Reasonable bounds
      implied_vol > 0 & implied_vol < 5
    )

  if(nrow(clean_data) == 0) {
    cat("ERROR: No data left after cleaning!\n")
    return(NULL)
  }

  smile_data <- clean_data %>%
    arrange(moneyness)

  cat("\n Smile Data (Sigma_i vs K/S) \n")
  cat("Data points:", nrow(smile_data), "\n")
  cat("IV range:", round(min(smile_data$implied_vol), 3), "to", round(max(smile_data$implied_vol), 3), "\n")

```

```

return(list(
  smile = smile_data,
  ticker = ticker,
  current_price = current_price
))
}

create_plots <- function(analysis_data) {

  if(is.null(analysis_data)) {
    cat("No data to plot\n")
    return()
  }

  smile_data <- analysis_data$smile
  ticker <- analysis_data$ticker

  par(mfrow = c(1, 2))

  if(nrow(smile_data) > 0) {
    finite_mask <- is.finite(smile_data$moneyness) & is.finite(smile_data$implied_vol)
    plot_smile <- smile_data[finite_mask, ]

    if(nrow(plot_smile) > 0) {
      plot(plot_smile$moneyness, plot_smile$implied_vol,
        main = paste(ticker, "Volatility Smile"),
        xlab = "Moneyness (K/S)", ylab = "Implied Volatility",
        pch = 16, cex = 0.8,
        col = ifelse(plot_smile$option_type == "call", "blue", "red"))

      abline(v = 1, lty = 2, col = "gray")
      legend("topright", c("Calls", "Puts"), col = c("blue", "red"), pch = 16, cex = 0.8)

      moneyness_regions <- cut(plot_smile$moneyness,
        breaks = seq(0.5, 2.0, 0.2),
        include.lowest = TRUE)
      region_counts <- table(moneyness_regions)
      cat("\nSmile data coverage:\n")
      print(region_counts[region_counts > 0])
    }
  }

}

result <- get_best_options_data()

## Selected ticker: AAPL
## Moneyness range: 0.546 to 1.613

if(!is.null(result)) {
  analysis_ready <- prepare_analysis_data(result$data, result$ticker)

  create_plots(analysis_ready)
}

```

```

if(!is.null(analysis_ready)) {
  cat("- Smile data:", nrow(analysis_ready$smile), "options\n")

  smile_data_final <- analysis_ready$smile
  term_structure_final <- analysis_ready$term_structure
  best_ticker_final <- analysis_ready$ticker

  cat("\nData saved as:\n")
  cat("- smile_data_final (for smile analysis)\n")
  cat("- best_ticker_final (ticker name)\n")
}
} else {
  cat("Data collection failed!\n")
}

```

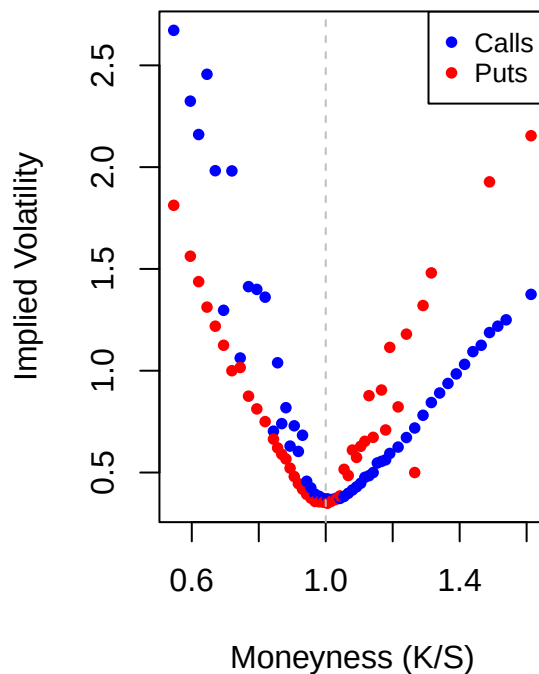
```

##
## Smile Data (Sigma_i vs K/S)
## Data points: 101
## IV range: 0.349 to 2.672

##
## Smile data coverage:
## moneyness_regions
## [0.5,0.7] (0.7,0.9] (0.9,1.1] (1.1,1.3] (1.3,1.5] (1.5,1.7]
##      12      20      32      23      10      4
## - Smile data: 101 options
##
## Data saved as:
## - smile_data_final (for smile analysis)
## - best_ticker_final (ticker name)

```

AAPL Volatility Smile



The following code filters for calls and saves the results to a csv file, which is submitted. I filtered for calls because there was more available data for calls. To be expected. Calls are easier to deal with than puts which involve selling stocks.

```
library(dplyr)
calls_data <- smile_data_final %>% filter(option_type == "call")

collection_date <- Sys.Date()
original_stock_price <- as.numeric(getQuote("AAPL")$Last)

print(paste("Collection date:", collection_date))

## [1] "Collection date: 2025-06-09"

print(paste("Current AAPL price:", original_stock_price))

## [1] "Current AAPL price: 201.45"

calls_data$stock_price <- original_stock_price
calls_data$collection_date <- as.character(collection_date)
calls_data$moneyness_check <- calls_data$Strike / calls_data$stock_price

write.csv(calls_data, "different_name_from_what_i_used.csv", row.names = FALSE)

data <- read.csv("project_dataset.csv", stringsAsFactors = FALSE)
print(length(data$moneyness))

## [1] 53
```

The data set used for the analysis has 53 observations and 24 columns. Important columns include the ContractID, the Strike Price, Expiration Date (note that all are the same), stock price (on June 7th) and implied_vol.

2) Fit Nonparametric Model

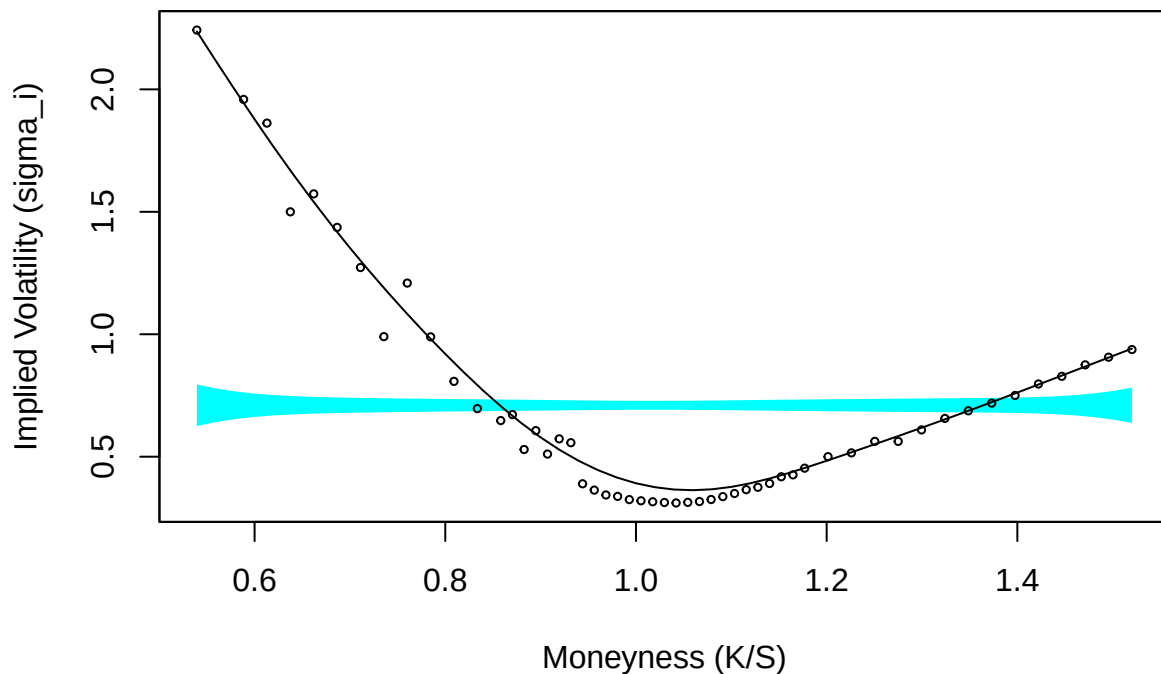
(a) Load Data

```
data <- read.csv("project_dataset.csv", stringsAsFactors = FALSE)
x <- data$moneyness
y <- data$implied_vol
```

(b) Fit Model

(i) Subjective Choice for Smoothing Parameter

```
sm.regression(x, y, h = 0.08, model = "no effect",
  xlab = "Moneyness (K/S)",
  ylab = "Implied Volatility (sigma_i)",
  main = "Test for No Effect: Volatility vs Moneyness")
```



```
## Test of no effect model:  significance = 0
```

(ii) Manual Search for Optimal Smoothing Parameter

We loop over a range of values for h and find the one with minimal CV score.

```
h_range <- diff(range(x)) / length(x)
h_candidates <- seq(h_range * 0.5, h_range * 3, length.out = 10)

cv_scores_manual <- numeric(length(h_candidates))

for(i in 1:length(h_candidates)) {
  h <- h_candidates[i]
  predictions <- numeric(length(x))

  for(j in 1:length(x)) {
```

```

x_train <- x[-j]
y_train <- y[-j]
x_test  <- x[j]

tryCatch({
  fit <- sm.regression(x_train, y_train, h = h, eval.points = x_test, display = "none")
  predictions[j] <- fit$estimate
}, error = function(e) {
  predictions[j] <- NA
})
}

valid_predictions <- !is.na(predictions)
if(sum(valid_predictions) > 0) {
  cv_scores_manual[i] <- mean((y[valid_predictions] - predictions[valid_predictions])^2)
} else {
  cv_scores_manual[i] <- Inf
}
}

optimal_idx_manual <- which.min(cv_scores_manual)
optimal_h_manual <- h_candidates[optimal_idx_manual]

cat("Manual method optimal bandwidth:", round(optimal_h_manual, 6), "\n")

## Manual method optimal bandwidth: 0.050375
cat("Manual method CV score:", round(cv_scores_manual[optimal_idx_manual], 6), "\n")

## Manual method CV score: 0.003791

sm.regression(x, y, h = optimal_h_manual, xlab = "Moneyness (K/S)",
              ylab = "Implied Volatility (sigma_i)",
              main = "Test for No Effect: Volatility vs Moneyness", model = "no effect", display = "none")

## Test of no effect model:  significance = 0

(iii) Model with Cross-Validation Smoothing Parameter

fit <- sm.regression(x, y, method = "cv", xlab = "Moneyness (K/S)",
                    ylab = "Implied Volatility (sigma_i)",
                    main = "Test for No Effect: Volatility vs Moneyness", model = "no effect", display = "none")

## Test of no effect model:  significance = 0
h1 <- fit$h # save h

```

(c) Metrics and Plots

(i) Metrics (CV Score)

We compute the cross-validation scores for all three models

```

cv_mse <- function(x, y, h) {
  n <- length(y)
  loo_preds <- sapply(seq_len(n), function(i) {
    xi <- x[i]

```

```

    x_rest <- x[-i]; y_rest <- y[-i]
    w <- dnorm((xi - x_rest)/h)
    sum(w * y_rest) / sum(w)
  })
  mean((y - loo_preds)^2)
}

cv_1 <- cv_mse(x, y, .08)
cv_2 <- cv_mse(x, y, h1)
cat("Subjective CV Criterion =", cv_1, "\n Default CV Criterion =", cv_2, "\n")

## Subjective CV Criterion = 0.01237763
## Default CV Criterion = 0.007804461

```

Method of Selecting h	CV Score
Subjective	.0123
Manual Search	0.0038
Default (method = "cv")	.0078

Since all three plots corresponding to the 3 methods of choosing h look similar, we pick the lowest CV criterion value (manual search) for the final model.

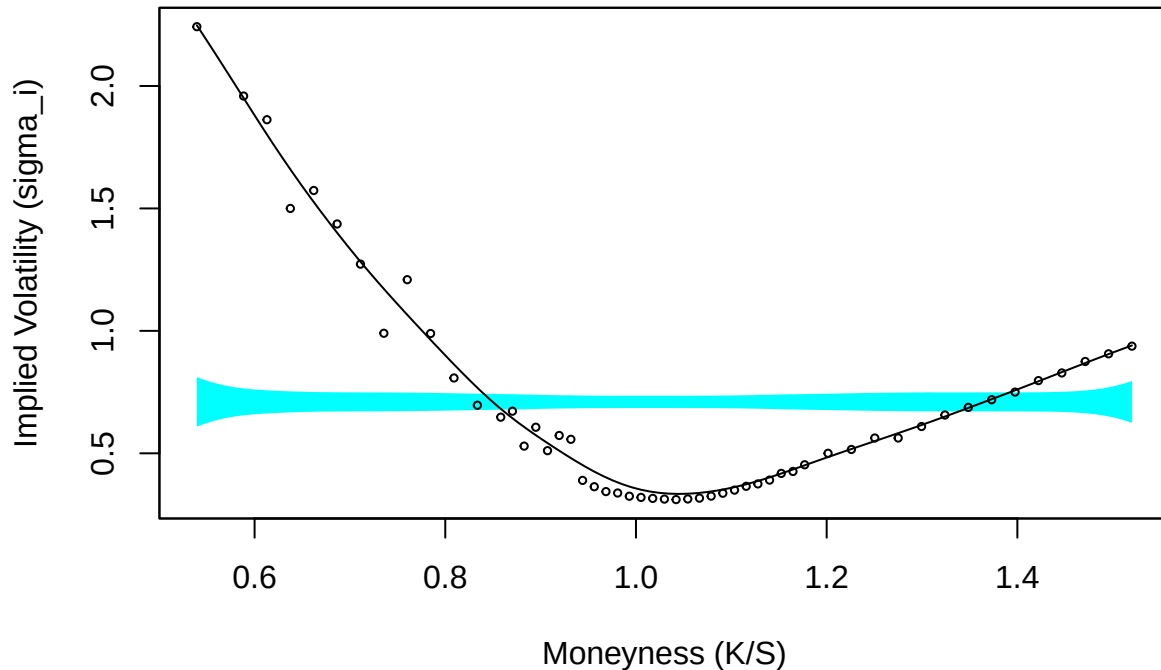
(ii) Fit Final Nonparametric Model

```

x_grid <- seq(min(x), max(x), length.out = 100)

final_model <- sm.regression(x, y,
  h = optimal_h_manual,
  eval.points = x_grid,
  xlab = "Moneyness (K/S)",
  ylab = "Implied Volatility (sigma_i)",
  main = "Test for No Effect: Volatility vs Moneyness",
  model = "no effect"
)

```



```
## Test of no effect model:  significance = 0
```

(iii) Properties of estimator

To summarize the properties of the estimator:

```
meaningful_points <- c(0.70, 0.85, 0.95, 1.00, 1.05, 1.15, 1.30)

meaningful_points <- meaningful_points[meaningful_points >= min(x) & meaningful_points <= max(x)]

meaningful_estimates <- sm.regression(x, y,
                                     h = optimal_h_manual,
                                     eval.points = meaningful_points,
                                     poly.index = 1,
                                     display = "none")

for(i in 1:length(meaningful_points)) {
  cat("m_hat(", meaningful_points[i], ") = ", round(meaningful_estimates$estimate[i], 4), "\n")
}

## m_hat( 0.7 ) = 1.3348
## m_hat( 0.85 ) = 0.7088
## m_hat( 0.95 ) = 0.4385
## m_hat( 1 ) = 0.3563
## m_hat( 1.05 ) = 0.3343
## m_hat( 1.15 ) = 0.4143
## m_hat( 1.3 ) = 0.6162
```

We see that $\hat{m}$ obtains a minimum close to 1 (strike = stock price) close to 1.05, as predicted by financial theory and the volatility shoots up for values farther from 1 (faster at low values). This makes sense in context as when the strike price is near the stock price is a low volatility “no arbitrage” situation.

(iv) Diagnostic Plots

```
fitted_at_obs <- approx(x_grid, final_model$estimate, xout = x)$y
residuals <- y - fitted_at_obs

ss_res <- sum(residuals^2, na.rm = TRUE)
ss_tot <- sum((y - mean(y))^2)
r_squared_nonparam <- 1 - ss_res / ss_tot
par(mfrow = c(2, 2))

plot(x, y,
     pch = 16, col = "blue", cex = 0.8,
     main = "Volatility Smile\n(Nonparametric Kernel Regression)",
     xlab = "Moneyness (K/S)",
     ylab = "Implied Volatility")

lines(x_grid, final_model$estimate, col = "red", lwd = 2)

abline(v = 1, lty = 2, col = "gray", lwd = 1)

legend("topright",
     c("Call Options", "Kernel Regression", "ATM"),
     col = c("blue", "red", "gray"),
     pch = c(16, NA, NA),
     lty = c(NA, 1, 2),
     lwd = c(NA, 2, 1),
     cex = 0.7)

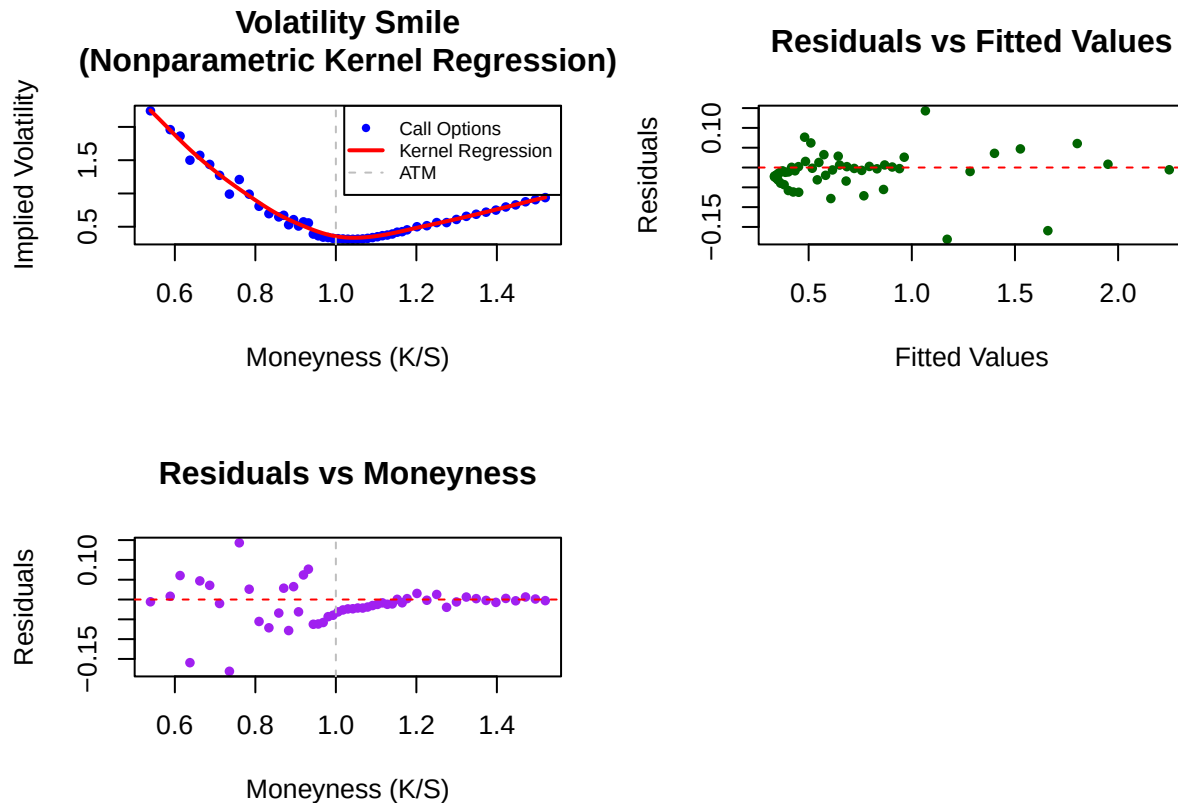
plot(fitted_at_obs, residuals,
     pch = 16, col = "darkgreen", cex = 0.8,
     main = "Residuals vs Fitted Values",
     xlab = "Fitted Values",
     ylab = "Residuals")

abline(h = 0, lty = 2, col = "red")

plot(x, residuals,
     pch = 16, col = "purple", cex = 0.8,
     main = "Residuals vs Moneyness",
     xlab = "Moneyness (K/S)",
     ylab = "Residuals")

abline(h = 0, lty = 2, col = "red")
abline(v = 1, lty = 2, col = "gray")

par(mfrow = c(1, 1))
```



The residuals show no clear pattern except that the estimator is more accurate when moneyness is greater than 1.

(v) DOF and Estimate Error Standard Deviation

```
h2df<- function(h_target, x, y){
  f<-function(df){
    sm.regression(x, y, df=df, display="none")$h - h_target
  }
  hmin<-sm.regression(x, y, df=length(x), display="none")$h
  hmax<-sm.regression(x, y, df=2.01, display="none")$h
  if ((h_target<= hmax)&(h_target >= hmin)){
    dfroot<-uniroot(f, c(2.01, length(x)), tol=0.005)$root
    return(round(dfroot, 2))
  }
  else return("out of range")
}
```

```
dof <- h2df(optimal_h_manual, x, y)
cat("Degrees of freedom: ", dof)
```

```
## Degrees of freedom: 9.38
```

Thus, the degrees of freedom corresponding to the estimate $\hat{m}(\cdot)$ is 9.38. To obtain an estimate of the error standard deviation we use the method based on second difference to obtain:

```
sm.sigma(x, y)$estimate
```

```
## [1] 0.05628304
```

(vi) Low Degree Polynomial Model

Since a low degree polynomial model might be appropriate based on the “U” shape, we fit a quadratic and a cubic model, centered at 1.

```
quad_model <- lm(y ~ I(x - 1) + I((x - 1)^2))
cubic_model <- lm(y ~ I(x - 1) + I((x - 1)^2) + I((x - 1)^3))

x_eval <- seq(min(x), max(x), length.out = 100)

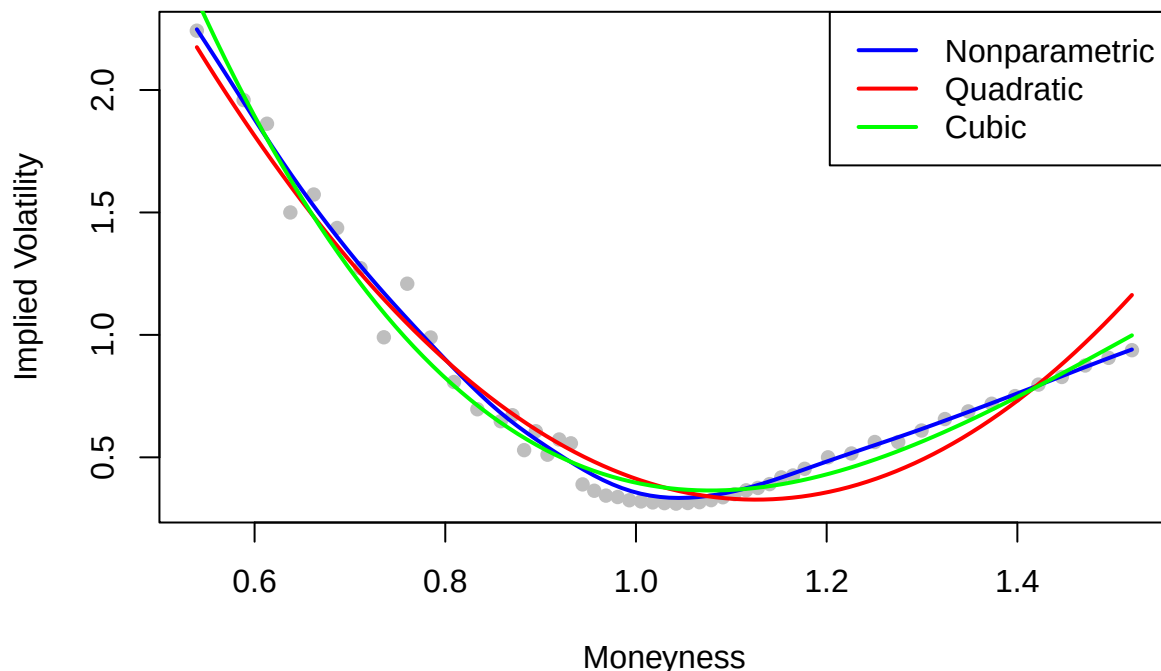
quad_pred <- predict(quad_model, newdata = data.frame(x = x_eval))
cubic_pred <- predict(cubic_model, newdata = data.frame(x = x_eval))

plot(x, y, pch = 16, col = "gray",
     main = "Nonparametric vs Polynomial Models",
     xlab = "Moneyness", ylab = "Implied Volatility")

lines(x_grid, final_model$estimate, col = "blue", lwd = 2) # Nonparametric
lines(x_eval, quad_pred, col = "red", lwd = 2)             # Quadratic
lines(x_eval, cubic_pred, col = "green", lwd = 2)          # Cubic

legend("topright",
      c("Nonparametric", "Quadratic", "Cubic"),
      col = c("blue", "red", "green"),
      lwd = 2)
```

Nonparametric vs Polynomial Models



```
nonparametric_results <- list(
  r_squared = r_squared_nonparam,
  rmse = sqrt(mean(residuals^2, na.rm = TRUE))
)
```

```

cat("R-squared comparison:\n")

## R-squared comparison:
cat("Quadratic:", round(summary(quad_model)$r.squared, 4), "\n")

## Quadratic: 0.9574
cat("Cubic:", round(summary(cubic_model)$r.squared, 4), "\n")

## Cubic: 0.9777
cat("Nonparametric:", round(nonparametric_results$r_squared, 4), "\n")

## Nonparametric: 0.9875

```

Based on a visual analysis and the R^2 values, our nonparametric model is a better fit.

(d) Analysis

Since all of the tests for no effect yield a p-value of 0 we reject the null hypothesis that implied volatility doesn't depend on moneyness. Thus strong evidence of a non-constant relationship. We also conclude that a nonparametric model is better suited than a low-degree polynomial. Based on the fitted estimator, we see the expected volatility smile, not quite centered at 1 (when the option's strike price equals stock price), as predicted by financial theorists, but approximately so (at ~1.1). In both directions, away from this central flat region, volatility increases as expected.

3) Semi-parametric Model

Based on financial theory and the previous result, we expect a “smile” shape. A U-shaped concave up curve is expected because in practice options are priced using computer simulations of a binomial tree model. Financial institutions hedge against risk by letting the volatility increase for values of moneyness far from 1 (if the strike price is very different from the stock price i.e. far out of the money or far in the money).

Thus, in this section we attempt to enhance the fit by using a semi-parametric model with a quadratic parametric component and a nonparametric component to capture the deviations of the true curve from this simple curve.

$$\sigma_i = \beta_1(M_i - 1)^2 + m(M_i) + \epsilon_i$$

(a) Estimate Regression Function

We estimate $m(s) = E(\sigma_i | M_i = s)$ in the above model.

```

data <- read.csv("project_dataset.csv", stringsAsFactors = FALSE)

calls_data <- data %>%
  filter(option_type == "call") %>%
  filter(!is.na(moneyness) & !is.na(implied_vol)) %>%
  filter(is.finite(moneyness) & is.finite(implied_vol)) %>%
  arrange(moneyness)

calls_data$moneyness_sq <- (calls_data$moneyness - 1)^2

semipar_bw <- npplregbw(implied_vol ~ moneyness_sq | moneyness,
  data = calls_data, regtype = "ll")

```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multi
semipar_fit <- npplreg(semipar_bw)

moneyness_grid <- seq(min(calls_data$moneyness), max(calls_data$moneyness), length.out = 100)
param_eval <- matrix(0, nrow = length(moneyness_grid), ncol = 1)
nonparam_component <- npplreg(semipar_bw,
                              exdat = param_eval,
                              ezdat = moneyness_grid)$mean
```

(b) DOF and Error Variance

We obtain the DOF and estimate the error variance from the previous part

```
h_semipar <- semipar_bw$bw$yzbw$bw

cat("Extracted bandwidth:", h_semipar, "\n")

## Extracted bandwidth: 0.05286996
h2df <- function(h_target, x, y){
  f <- function(df){
    sm.regression(x, y, df=df, display="none")$h - h_target
  }
  hmin <- sm.regression(x, y, df=length(x), display="none")$h
  hmax <- sm.regression(x, y, df=2.01, display="none")$h
  if ((h_target <= hmax) & (h_target >= hmin)){
    dfroot <- uniroot(f, c(2.01, length(x)), tol=0.005)$root
    return(round(dfroot, 2))
  }
  else return("out of range")
}

df_nonparametric <- h2df(h_semipar, calls_data$moneyness, calls_data$implied_vol)
df_parametric <- 2
df_total <- df_parametric + df_nonparametric

residuals_semipar <- residuals(semipar_fit)
error_variance <- var(residuals_semipar)

cat("Bandwidth (h):", round(h_semipar, 6), "\n")

## Bandwidth (h): 0.05287
cat("Degrees of freedom - Parametric:", df_parametric, "\n")

## Degrees of freedom - Parametric: 2
cat("Degrees of freedom - Nonparametric:", df_nonparametric, "\n")

## Degrees of freedom - Nonparametric: 9.02
cat("Total degrees of freedom:", df_total, "\n")

## Total degrees of freedom: 11.02
cat("Error variance estimate:", round(error_variance, 4), "\n")

## Error variance estimate: 0.0025
```

(c) Linearity Test

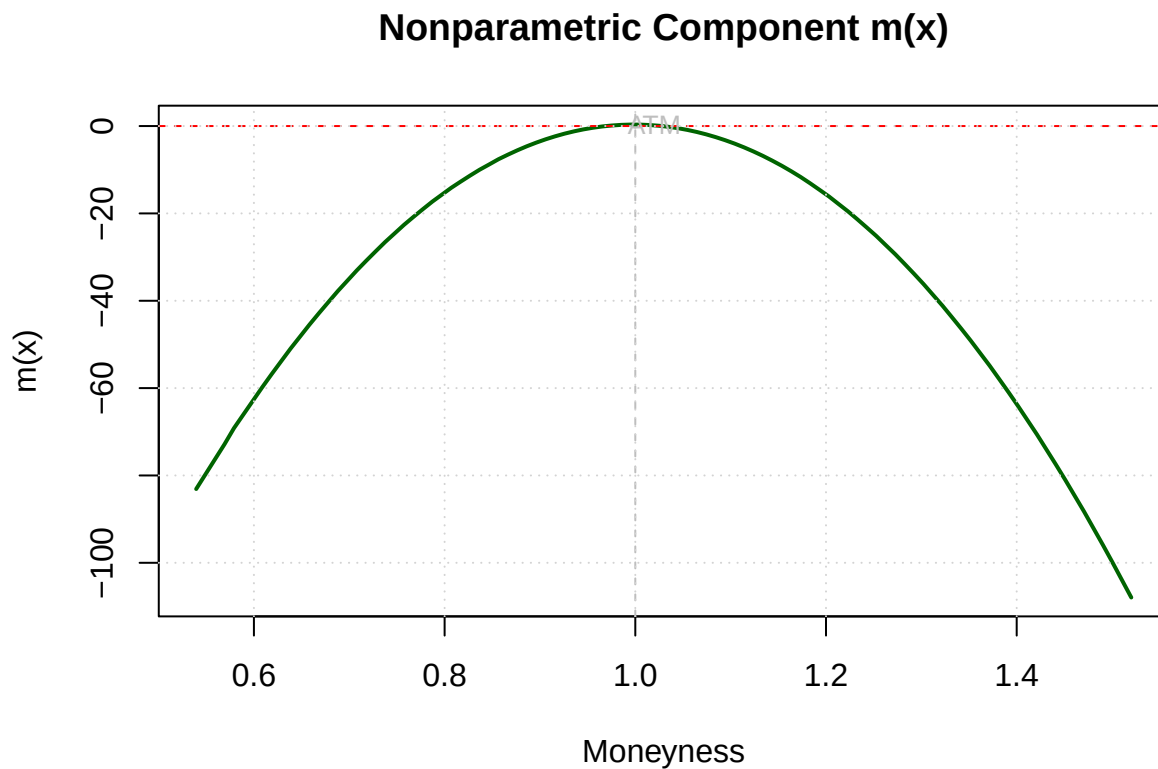
We plot $\hat{m}$ and look for linearity

```
plot(moneyness_grid, nonparam_component,
     type = "l", lwd = 2, col = "darkgreen",
     main = "Nonparametric Component m(x)",
     xlab = "Moneyness", ylab = "m(x)")

abline(v = 1, lty = 2, col = "gray")      # ATM line
abline(h = 0, lty = 2, col = "red")       # Zero line

grid(col = "lightgray", lty = 3)

text(1.02, max(nonparam_component) * 0.9, "ATM", col = "gray", cex = 0.8)
```



It does not appear linear, but the negative values suggest that the nonparametric component drags the parabola downwards.

(d) Semi-parametric regression model

```
beta_1 <- coef(semipar_fit)[1]
se_beta_1 <- coef(semipar_fit, errors = TRUE)[1]

cat("Semiparametric Model: sigma = Beta_1(M-1)^2 + m(M) + eps\n\n")

## Semiparametric Model: sigma = Beta_1(M-1)^2 + m(M) + eps

cat("Parameter Estimates:\n")

## Parameter Estimates:
```

```

cat("Beta_1 (quadratic coef):", round(beta_1, 6), "\n")

## Beta_1 (quadratic coef): 402.3456
cat("\nStandard Errors:\n")

##
## Standard Errors:
cat("SE(Beta_1):", round(se_beta_1, 6), "\n")

## SE(Beta_1): 221.459
ci_beta_1 <- c(beta_1 - 1.96*se_beta_1, beta_1 + 1.96*se_beta_1)
cat("\n95% Confidence Interval:\n")

##
## 95% Confidence Interval:
cat("Beta_1: (", round(ci_beta_1[1], 4), ",", round(ci_beta_1[2], 4), ")\n")

## Beta_1: ( -31.7139 , 836.4052 )

Now, we plot the model.
fitted_values <- fitted(semipar_fit)

plot(calls_data$moneyness, calls_data$implied_vol,
     pch = 16, col = "blue", cex = 0.8,
     main = "Semiparametric Volatility Smile\n sigma = Beta_1(M-1)^2 + m(M) + eps",
     xlab = "Moneyness (K/S)",
     ylab = "Implied Volatility")

points(calls_data$moneyness, fitted_values,
       col = "red", pch = 16, cex = 0.6)

order_idx <- order(calls_data$moneyness)
lines(calls_data$moneyness[order_idx], fitted_values[order_idx],
      col = "red", lwd = 2)

abline(v = 1, lty = 2, col = "gray")

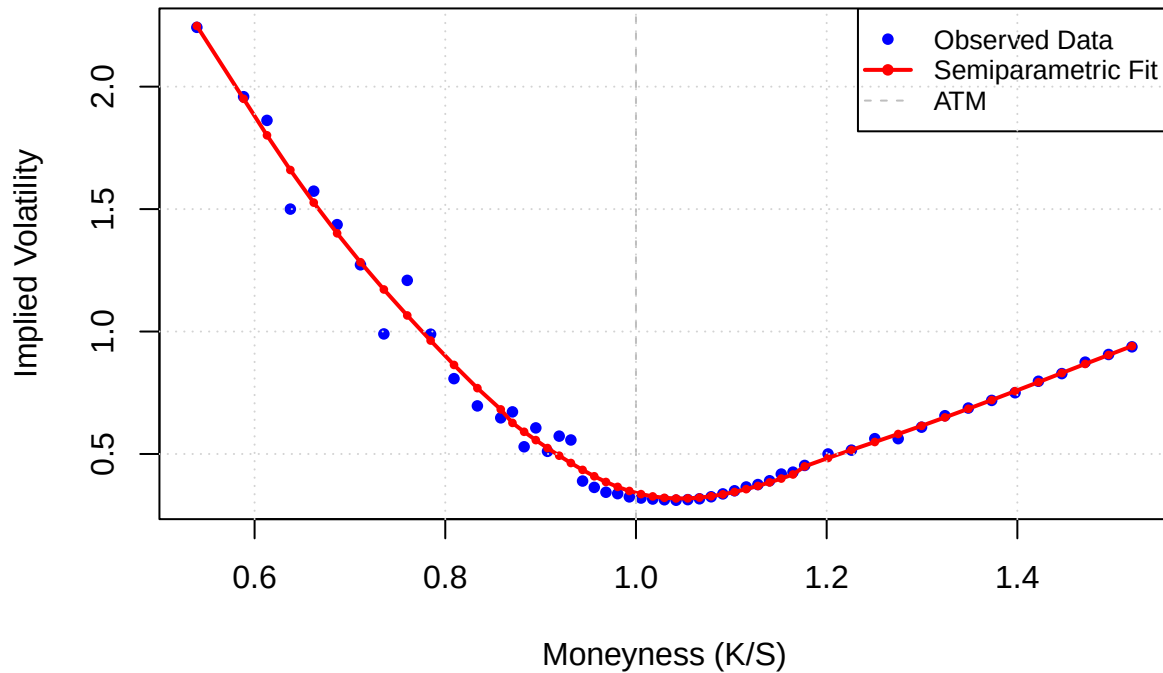
legend("topright",
      c("Observed Data", "Semiparametric Fit", "ATM"),
      col = c("blue", "red", "gray"),
      pch = c(16, 16, NA),
      lty = c(NA, 1, 2),
      lwd = c(NA, 2, 1),
      cex = 0.8)

grid(col = "lightgray", lty = 3)

```

Semiparametric Volatility Smile

$\sigma = \text{Beta}_1(M-1)^2 + m(M) + \text{eps}$



And see that we obtain a similar curve with a perhaps slightly better fit

(e) Diagnostic plots

```
residuals_semipar <- residuals(semipar_fit)
fitted_values <- fitted(semipar_fit)

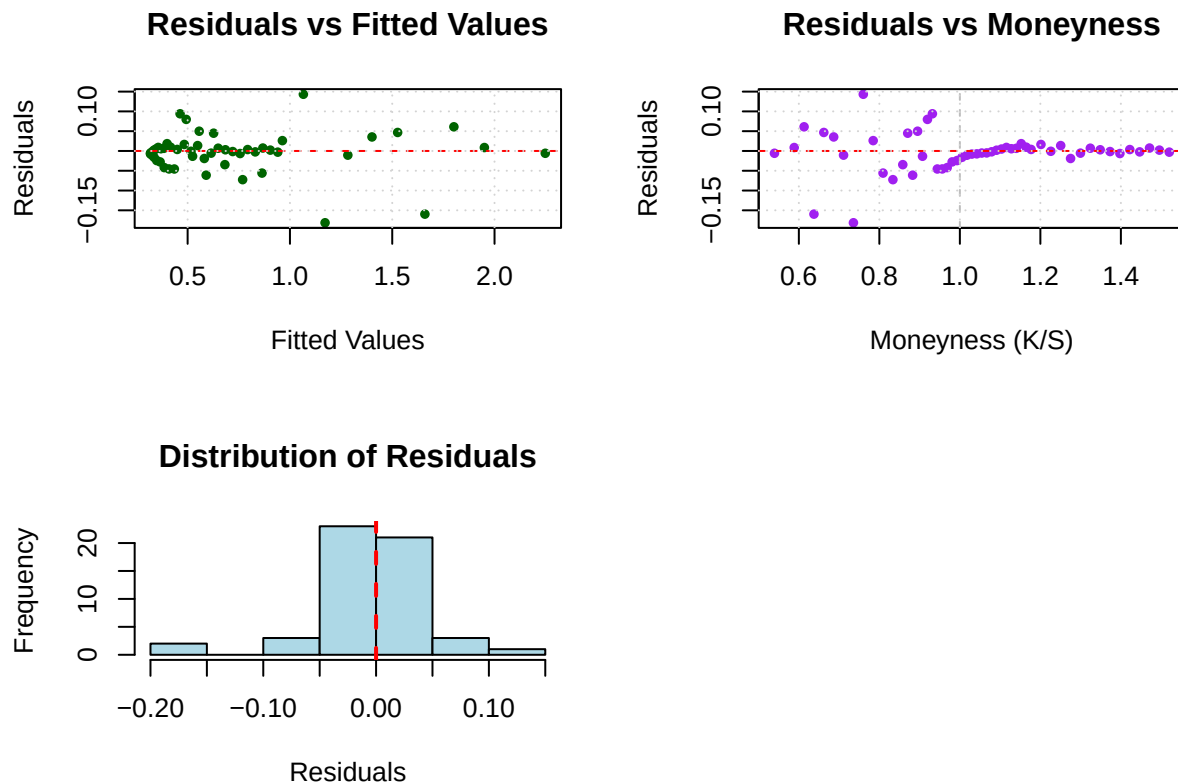
par(mfrow = c(2, 2))

plot(fitted_values, residuals_semipar,
     pch = 16, col = "darkgreen", cex = 0.8,
     main = "Residuals vs Fitted Values",
     xlab = "Fitted Values", ylab = "Residuals")
abline(h = 0, lty = 2, col = "red")
grid(col = "lightgray", lty = 3)

plot(calls_data$moneyness, residuals_semipar,
     pch = 16, col = "purple", cex = 0.8,
     main = "Residuals vs Moneyness",
     xlab = "Moneyness (K/S)", ylab = "Residuals")
abline(h = 0, lty = 2, col = "red")
abline(v = 1, lty = 2, col = "gray")
grid(col = "lightgray", lty = 3)

hist(residuals_semipar, breaks = 10, col = "lightblue", border = "black",
     main = "Distribution of Residuals",
     xlab = "Residuals", ylab = "Frequency")
abline(v = 0, lty = 2, col = "red", lwd = 2)
```

```
par(mfrow = c(1, 1))
```



Residuals look similar to nonparametric case. Same comments apply.

Lastly, let's compare the CV criteria (MSE) in the nonparametric vs semi-parametric case.

```
cv_semiparametric <- mean(residuals(semipar_fit)^2)
print(cv_semiparametric)
```

```
## [1] 0.002447335
```

The CV score is

```
0.002447335
```

lower than the chosen value of .0038.

(f) Discussion

Based on the plots, analysis and CV score we conclude that the semi-parametric method works slightly better, but is more difficult to execute, so the nonparametric approach is most practical.